Министерство просвещения Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«ХЕРСОНСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА

по дисциплине
«Современные операционные системы и базы данных»

Тема «Подключение базы данных к сайту-портфолио»

Направление подготовки
09.04.04 «Программная инженерия»

Составитель: доцент, к.н., Галлини Надежда Игоревна

Ялта
2026

СОДЕРЖАНИЕ

Цель и результат работы 3

1 Архитектура приложения 4

2 Подготовка и первый запуск 5

3 Таблицы и ограничения базы 6

4 Создание и публикация проекта 7

5 Чтение данных из PowerShell и Node.js 8

6 Запись данных и транзакция 9

7 От формы админки до SQL 10

8 Подключение своего портфолио 11

9 Заявки и административные статусы 12

10 Проверки и сохранность данных 13

11 Связь с IIS и самостоятельное задание 14

12 Отчёт и критерии оценивания 15

Список использованных источников 16

Цель и результат работы

Цель работы: подключить SQLite к серверной части сайта-портфолио, изучить структуру таблиц и параметризованные SQL-запросы, организовать управление материалами через административный интерфейс и проверить сохранность данных.

Работа продолжает занятия по CMD, PowerShell, Windows Terminal и разработке портфолио. Образец – серверная версия сайта «Сайт срочно». В качестве учебного комплекта используется отдельная копия приложения на Node.js 24 и SQLite. Изменять действующий сайт преподавателя и его базу запрещено.

Результат: сайт получает проекты из БД, администратор создаёт, редактирует и публикует материалы; после перезапуска сервера записи сохраняются. Студент объясняет, как действие в браузере превращается в HTTP-запрос, SQL-запрос и изменение записи.

Потребуются Node.js 24 LTS, современный браузер и редактор кода. SQLite входит в используемый модуль node:sqlite, отдельная служба СУБД не устанавливается. API node:sqlite зависит от версии Node.js; комплект проверен на версии 24.16.0 [1]. Установка IIS, публичный IP и домен для локального этапа не нужны.

Приложен архив «Портфолио БД учебный комплект.zip». В нём находятся сервер, админка и минимальная главная страница. Студенты получают собственную пустую базу. Материалы реального сайта и пользовательские заявки в учебный архив не включены.

1 Архитектура приложения

Браузер не подключается к файлу БД напрямую. Он отправляет запрос серверу Node.js. Сервер проверяет права и данные, выполняет SQL и возвращает результат. Схема взаимодействия: браузер → HTTP API → сервер → SQLite → ответ браузеру. Назначение компонентов приведено в таблице 1.

Таблица 1 – Компоненты учебного приложения

Файл или компонент	**Назначение**
index.html и assets/	Публичная страница и её оформление
server/admin.html, admin.js, admin.css	Форма входа, список материалов, редактор и заявки
server/app.mjs	HTTP-маршруты, проверка доступа и выдача страниц
server/store.mjs	Открытие БД и сохранение материалов
server/schema.sql	Создание таблиц и ограничений
server/inspect.mjs	Просмотр записей из консоли без изменения БД
portfolio.sqlite	Файл базы в отдельном каталоге вне сайта

Публичный запрос получает только опубликованные материалы. Администратор после входа видит также черновики. В этой версии один администратор; пароль хранится в виде scrypt-хеша с солью. Сеанс использует cookie HttpOnly и завершается при выходе, истечении срока или перезапуске сервера.

Задание: найдите в app.mjs маршрут /api/admin/entries и функцию auth. Укажите, где заканчивается работа браузера и начинается выполнение SQL. Объясните, почему пароль нельзя вставлять в admin.js.

2 Подготовка и первый запуск

Распакуйте учебный архив в папку portfolio-db внутри своего профиля. Проверьте, что index.html и server находятся непосредственно в ней. Выполните листинг 1 в PowerShell. Если node не найден, установку Node.js организует преподаватель до начала работы.

Листинг 1 – Проверка среды и выбор учебной базы

```
node --version
Set-Location "$env:USERPROFILE\portfolio-db"
$env:PORTFOLIO_DATA = Join-Path $env:LOCALAPPDATA `
    'PortfolioDbLab'
$env:PORT = '3090'
Get-ChildItem .\server
```

PORTFOLIO_DATA задаёт каталог учебных данных. Не указывайте каталог рабочего сайта или общую базу группы. Для нескольких независимых работ используйте разные имена папок данных. Значения переменных окружения действуют в текущем сеансе.

При первом запуске задайте пароль длиной не менее 14 символов по листингу 2. Пароль вводится скрыто. Команды не требуют сохранения пароля в файле; после старта база содержит хеш. При следующих запусках существующей БД пароль в ADMIN_PASSWORD не нужен.

Листинг 2 – Создание пароля первого запуска и запуск сервера

```
$secret = Read-Host 'Пароль администратора' -AsSecureString
$ptr = [Runtime.InteropServices.Marshal]::SecureStringToBSTR($secret)
try {
  $env:ADMIN_PASSWORD =
    [Runtime.InteropServices.Marshal]::PtrToStringBSTR($ptr)
} finally {
  [Runtime.InteropServices.Marshal]::ZeroFreeBSTR($ptr)
}
try { node .\server\app.mjs }
finally { Remove-Item Env:ADMIN_PASSWORD -ErrorAction SilentlyContinue }
```

Откройте http://127.0.0.1:3090/admin/ и войдите. Окно PowerShell оставьте открытым; Ctrl+C останавливает сервер. Если порт занят, измените PORT и адрес браузера одинаково. В учебной базе список материалов сначала пуст.

3 Таблицы и ограничения базы

Изучите server/schema.sql. В таблице 2 приведены сущности. В учебной модели проекты, услуги и статьи хранятся вместе: поле kind определяет тип. Такой выбор сокращает повторяющиеся поля, но требует проверки допустимых значений.

Таблица 2 – Сущности базы данных

Таблица	Назначение и ключ
entries	Материалы; первичный ключ id, уникальная пара kind и slug
admins	Одна учётная запись; id=1, соль и хеш пароля
inquiries	Заявки; id, контакт, сообщение, согласие и статус

Основные поля entries: title – название, description – описание карточки, body – полный текст, image – путь изображения, url – ссылка на проект, tech – строка технологий, published – признак публикации, sort_order – порядок, version – версия записи.

Листинг 3 – Фрагмент схемы entries из schema.sql

```
CREATE TABLE IF NOT EXISTS entries (
  id INTEGER PRIMARY KEY,
  kind TEXT NOT NULL
    CHECK (kind IN ('project','service','article')),
  slug TEXT NOT NULL,
  title TEXT NOT NULL,
  description TEXT NOT NULL DEFAULT '',
  body TEXT NOT NULL DEFAULT '',
  image TEXT NOT NULL DEFAULT '',
  url TEXT NOT NULL DEFAULT '',
  tech TEXT NOT NULL DEFAULT '',
  published INTEGER NOT NULL DEFAULT 0
    CHECK (published IN (0,1)),
  sort_order INTEGER NOT NULL DEFAULT 0,
  version INTEGER NOT NULL DEFAULT 1,
  updated_at TEXT NOT NULL DEFAULT
    (strftime('%Y-%m-%dT%H:%M:%fZ','now')),
  UNIQUE(kind, slug)
);
```

Листинг 3 объясняет уже созданную таблицу; отдельно выполнять его не требуется. Найдите в файле схемы индекс entries_public. Объясните назначение PRIMARY KEY, NOT NULL, DEFAULT, CHECK и UNIQUE [2].

4 Создание и публикация проекта

В админке выберите «Проекты» → «Добавить материал». Укажите название своей учебной работы, адрес my-first-project, краткое описание, подробный текст и технологии. Адрес вводится латиницей через дефис. Поля изображения и внешней ссылки можно пока оставить пустыми.

Сохраните без отметки «Опубликовано». Материал появится со статусом «Черновик». Откройте /portfolio/ в отдельной вкладке: карточка не должна отображаться. Прямой адрес /portfolio/my-first-project/ также не должен выдавать черновик.

Снова откройте материал, отметьте «Опубликовано» и сохраните. Обновите /portfolio/: карточка должна появиться. Перейдите на страницу проекта. Измените название в админке, сохраните и проверьте результат после обновления публичной страницы.

Создайте ещё два проекта и задайте sort_order: 10, 20 и 30. Меньшее значение показывается раньше. При одинаковом значении используется id. Поиск в админке позволяет найти запись по названию или адресу.

Тип и адрес существующего материала не меняются, чтобы сохранить ссылки. Для скрытия используйте снятие с публикации. Физическое удаление через админку намеренно не реализовано: запись остаётся в базе и может быть опубликована снова.

Аналогично создайте одну учебную услугу и одну статью. Они появятся в /services/ и /blog/. В учебной модели интерфейс использует общие поля; поле «Технологии» для статьи можно оставить пустым.

Проверка результата: три проекта, одна услуга и одна статья существуют в админке; публичная часть показывает только отмеченные материалы. Снимите публикацию одной записи и объясните, почему это UPDATE, а не DELETE.

5 Чтение данных из PowerShell и Node.js

Не закрывая сервер, откройте вторую вкладку PowerShell. Снова задайте путь к учебной базе и запустите inspect.mjs по листингу 4. Этот сценарий открывает БД только для чтения; пароль и заявки он не выводит.

Листинг 4 – Просмотр базы в отдельной вкладке

```
Set-Location "$env:USERPROFILE\portfolio-db"
$env:PORTFOLIO_DATA = Join-Path $env:LOCALAPPDATA `
    'PortfolioDbLab'
node .\server\inspect.mjs
```

Сопоставьте id, slug, published и version с админкой. Отредактируйте запись и повторите команду: её version увеличится. При остановке сервера файл БД остаётся на диске.

Создайте server/read-projects.mjs по листингу 5 и запустите node .\server\read-projects.mjs. Файл использует ту же переменную PORTFOLIO_DATA. Здесь показаны открытие SQLite, подготовка SQL и передача параметра отдельно от текста запроса [1].

Листинг 5 – Чтение опубликованных проектов из SQLite

```
import { DatabaseSync } from 'node:sqlite';
import { join } from 'node:path';
const folder = process.env.PORTFOLIO_DATA;
if (!folder) throw new Error('Задайте PORTFOLIO_DATA');
const db = new DatabaseSync(
  join(folder, 'portfolio.sqlite'), { readOnly: true });
const query = db.prepare(`
  SELECT id, title, slug, sort_order
  FROM entries
  WHERE kind = ? AND published = 1
  ORDER BY sort_order, id
`);
console.table(query.all('project'));
db.close();
```

Замените параметр project на article и повторите чтение. Затем уберите условие published=1 только в собственной учебной копии сценария и объясните изменение результата. Публичный маршрут сайта изменять для этого опыта не нужно.

6 Запись данных и транзакция

Создайте server/sql-experiment.mjs по листингу 6. Он вставляет учебный черновик, изменяет его и откатывает обе операции. Используйте только собственную учебную базу. Имя SQL эксперимент не должно совпадать с реальными проектами.

Листинг 6 – INSERT и UPDATE с последующим откатом

```
import { DatabaseSync } from 'node:sqlite';
import { join } from 'node:path';
const folder = process.env.PORTFOLIO_DATA;
if (!folder) throw new Error('Задайте PORTFOLIO_DATA');
const db = new DatabaseSync(join(folder, 'portfolio.sqlite'));
db.exec('BEGIN');
try {
  const result = db.prepare(`
    INSERT INTO entries(kind, slug, title)
    VALUES (?, ?, ?)
  `).run('project', 'sql-experiment', 'SQL эксперимент');
  const id = Number(result.lastInsertRowid);
  db.prepare(`
    UPDATE entries SET title = ?, version = version + 1
    WHERE id = ?
  `).run('Название изменено', id);
  console.table(db.prepare(
    'SELECT id, title, published FROM entries WHERE id = ?'
  ).all(id));
} finally {
  db.exec('ROLLBACK');
  db.close();
}
```

Запустите node .\server\sql-experiment.mjs. Во время транзакции сценарий видит вставленную запись. После завершения inspect.mjs и админка не должны её показывать. Объясните отличие ROLLBACK от COMMIT [3].

Знаки ? обозначают параметры. Пользовательский текст передаётся через run, а не добавляется в SQL конкатенацией. Название с апострофом должно сохраняться как текст и не менять смысл SQL-запроса. Найдите такую же передачу параметров в store.mjs.

7 От формы админки до SQL

Найдите обработчик entry-form в admin.js. Он собирает значения полей и вызывает API. В app.mjs сервер проверяет сеанс, CSRF-токен, обязательные поля, длину строк и допустимость ссылок. Затем saveEntry из store.mjs выполняет запрос к базе.

Таблица 3 – Основные маршруты приложения

Запрос	**Назначение**
POST /api/login	Проверить пароль и создать сеанс
GET /api/admin/entries	Получить материалы после входа
POST /api/admin/entries	Создать материал
PUT /api/admin/entries/ID	Изменить материал с проверкой version
GET /api/projects	Получить опубликованные проекты
POST /api/inquiries	Сохранить заявку с согласием
PUT /api/admin/inquiries/ID	Изменить статус заявки

Маршруты таблицы 3 исследуйте в средствах разработчика браузера, вкладка Network. Откройте редактор, измените описание и сохраните. Найдите PUT-запрос: сравните отправленные title, published, version с ответом сервера. Cookie и CSRF-токен в отчёт не копируйте.

Проверка конкурентного изменения: откройте одну запись в двух вкладках. В первой измените название и сохраните. Во второй измените описание и сохраните старую версию. Сервер должен вернуть 409, а форма – пояснение о конфликте. Обновите список и повторите правку по актуальной записи.

Самостоятельно найдите WHERE id=? AND version=? в store.mjs. Объясните, почему запрос не должен обновлять запись только по id, если пользователь редактирует устаревшие данные.

8 Подключение своего портфолио

Сначала сохраните копию своего статического портфолио. Для учебного подключения перенесите его index.html и папку assets в корень учебного комплекта, сохранив папку server. Сервер разрешает публичную выдачу файлов из assets; CSS, JavaScript и изображения поместите внутрь неё и исправьте ссылки.

В месте вывода карточек добавьте единственный контейнер из листинга 7. Сервер найдёт его и заменит содержимое карточками опубликованных проектов из БД. Старые статические карточки внутри этого блока удалите из учебной копии после проверки резервной копии.

Листинг **7** – Место для проектов в index.html

```
<section id="projects">
  <h2>Мои проекты</h2>
  <div class="project-grid"></div>
</section>
```

В этом варианте карточки формируются на сервере при загрузке страницы. Для получения JSON клиентским JavaScript предусмотрен /api/projects. Это два способа использования одной БД; для обязательной части достаточно серверного вывода.

Листинг 8 – Минимальные стили карточек в assets/style/style.css

```
.project-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 24px;
}
.project-card {
  border: 1px solid #ddd;
  border-radius: 16px;
  padding: 20px;
  min-width: 0;
}
.project-card img { max-width: 100%; }
.project-card a { overflow-wrap: anywhere; }
```

Используйте листинг 8, если в вашем CSS ещё нет этих классов. Открывайте сайт по http://127.0.0.1:3090/, а не двойным щелчком по index.html. При file:// сервер не обрабатывает запрос и БД не участвует в формировании страницы.

9 Заявки и административные статусы

В учебном комплекте запись заявки можно проверить HTTP-запросом без создания полноценной формы. Выполните листинг 9 во второй вкладке PowerShell. Используйте вымышленные учебные данные. Реальная отправка почты и Telegram в этой версии не выполняется.

Листинг 9 – Тестовое сохранение заявки через HTTP API

```
$data = @{
  name = 'Учебный пользователь'
  contact = 'student@example.com'
  service = 'Портфолио'
  budget = ''
  message = 'Тестовая заявка для практической работы'
  consent = $true
} | ConvertTo-Json
Invoke-RestMethod -Uri 'http://127.0.0.1:3090/api/inquiries' `
  -Method Post -ContentType 'application/json; charset=utf-8' `
  -Body ([Text.Encoding]::UTF8.GetBytes($data))
```

Откройте «Заявки» в админке. Проверьте контакт, текст и время. Измените статус с «Новая» на «В работе», затем на «Завершена». После обновления страницы выбранное значение должно сохраняться.

Измените consent на $false и повторите запрос: сервер должен отказать в сохранении. Наличие флажка в браузере недостаточно, сервер обязан проверять значение самостоятельно. Не создавайте большое количество одинаковых заявок: действует ограничение частоты.

В рабочем сайте форма использует server/site.js и этот же маршрут, когда сайт запущен через новую серверную часть. Сохранение записи в БД не является подтверждением отправки email. Уведомления подключаются отдельным серверным модулем с секретами вне браузера.

10 Проверки и сохранность данных

Проведите проверки по таблице 4 и запишите фактические результаты. Используйте только собственные записи. Для проверки перезапуска остановите сервер Ctrl+C, запустите снова с прежним PORTFOLIO_DATA и повторно войдите в админку.

Таблица **4** – Матрица проверок

Действие	**Ожидаемый результат**
Сохранить и перезапустить сервер	Запись сохранилась, требуется новый вход
Создать черновик	В админке виден, в публичном разделе отсутствует
Создать дубликат kind и slug	Ошибка 409; второй записи нет
Сохранить пустое название	Форма или сервер сообщает об ошибке
Сохранить ссылку javascript:	Сервер отклоняет ссылку
Сохранить текст с <script>	Текст не выполняется как JavaScript
Открыть API админки без входа	Ответ 401
Сохранить устаревшую version	Ответ 409; прежняя правка сохранена
Снять публикацию	Страница материала не выдаёт содержимое

Для простой учебной резервной копии остановите сервер и скопируйте весь каталог PORTFOLIO_DATA в отдельную папку вне сайта. SQLite использует режим WAL: при работающей БД одного файла .sqlite может быть недостаточно. Для резервирования без остановки нужен специальный механизм SQLite backup [4].

Не храните БД, пароли и резервные копии в assets. При необходимости восстановления используйте копию каталога данных и новое значение PORTFOLIO_DATA, сохраняя исходную базу.

11 Связь с IIS и самостоятельное задание

IIS самостоятельно не выполняет код Node.js. Для публичного размещения Node.js работает отдельным процессом, а IIS с URL Rewrite и ARR передаёт ему запросы. Проксировать нужно весь сайт, иначе HTML останется статическим, даже если API уже подключён [5].

В рамках этой работы достаточно локального запуска. Развёртывание как службы Windows, настройки ARR, привязки HTTPS и сертификат выполняются отдельным этапом преподавателем или администратором. Включать публичный доступ к учебной базе для защиты работы не требуется.

Самостоятельное задание 1. Добавьте ещё один проект со своим описанием и изображением в assets. Измените порядок, снимите и восстановите публикацию. Зафиксируйте одну и ту же запись в интерфейсе, SQL-выборке и публичной странице.

Самостоятельное задание 2. Измените свою копию read-projects.mjs: выведите количество проектов и количество опубликованных проектов отдельными запросами SELECT COUNT(*). Сравните числа с админкой.

Самостоятельное задание 3. Объясните недостатки хранения технологий в строке tech. Предложите таблицы technologies и entry_technologies с первичными и внешними ключами. Реализация связи многие-ко-многим является дополнительным заданием, а не скрытым требованием готовой схемы.

Дополнительное задание. Добавьте поле учебного года к материалу: опишите изменение схемы, чтения, валидации, формы и публичного вывода. Не удаляйте существующую базу для добавления поля: подготовьте миграцию ALTER TABLE и её повторяемую проверку.

12 Отчёт и критерии оценивания

Отчёт содержит цель, схему взаимодействия компонентов, описание таблиц, команды запуска, собственные SQL-запросы, результаты матрицы проверок и вывод. Приложите созданные read-projects.mjs и sql-experiment.mjs. Пароль, cookie, реальные заявки и файл рабочей БД в отчёт не включайте.

Нужны снимки трёх состояний одной записи: черновик в админке, опубликованная карточка на сайте, SQL-выборка с её id. Дополнительно покажите результат отката транзакции и конфликт двух вкладок. Каждый снимок должен иметь название и пояснение.

Основной текст оформляется Times New Roman 14 пт, интервал 1,5. Названия таблиц размещаются по центру, названия листингов – по правому краю, без жирного начертания. Код остаётся редактируемым. Критерии приведены в таблице 5.

Таблица **5** – Критерии оценивания

Критерий	**Баллы**
Локальный запуск и отдельная учебная БД	1
SQL-выборки, запись с параметрами и откат транзакции	1
Связь админки с публичным портфолио и самостоятельные задания	1
Проверки, доказательства сохранности данных и отчёт	1
Защита: объяснение – 0,5; изменение и проверка – 0,5	1
Итого	5

Вопросы для защиты: почему браузер не открывает SQLite напрямую; чем черновик отличается от удалённой записи; зачем нужны параметры SQL; какую проблему решает version; почему cookie не заменяет проверку прав; почему file:// не показывает данные БД; что необходимо добавить между IIS и Node.js.

Список использованных источников

1. SQLite // Node.js : документация. – URL: https://nodejs.org/docs/latest-v24.x/api/sqlite.html (дата обращения: 20.09.2026). – Текст : электронный.

2. CREATE TABLE // SQLite : документация. – URL: https://www.sqlite.org/lang_createtable.html (дата обращения: 20.09.2026). – Текст : электронный.

3. Transaction // SQLite : документация. – URL: https://www.sqlite.org/lang_transaction.html (дата обращения: 20.09.2026). – Текст : электронный.

4. SQLite Backup API // SQLite : документация. – URL: https://www.sqlite.org/backup.html (дата обращения: 20.09.2026). – Текст : электронный.

5. Reverse Proxy with URL Rewrite v2 and Application Request Routing // Microsoft Learn : сайт. – URL: https://learn.microsoft.com/en-us/iis/extensions/url-rewrite-module/reverse-proxy-with-url-rewrite-v2-and-application-request-routing (дата обращения: 20.09.2026). – Текст : электронный.